

Document |  
Roadmap |  
Twitter |  
Discord |  
Demo

## Table of Contents

- [What is RAGFlow?](#what-is-ragflow)
- [Demo](#demo)
- [Latest Updates](#latest-updates)
- [Key Features](#key-features)
- [System Architecture](#system-architecture)
- [Get Started](#get-started)
- [Configurations](#configurations)
- [Build a Docker image](#build-a-docker-image)
- [Launch service from source for development](#launch-service-from-source-for-development)
- [Documentation](#documentation)
- [Roadmap](#roadmap)
- [Community](#community)
- [Contributing](#contributing)

## What is RAGFlow?

[RAGFlow](https://ragflow.io/) is a leading open-source Retrieval-Augmented Generation (RAG) engine that fuses cutting-edge RAG with Agent capabilities to create a superior context layer for LLMs. It offers a streamlined RAG workflow adaptable to enterprises of any scale. Powered by a converged context engine and pre-built agent templates, RAGFlow enables developers to transform complex data into high-fidelity, production-ready AI systems with exceptional efficiency and precision.

## Demo

Try our demo at [https://demo.ragflow.io](https://demo.ragflow.io).

## Latest Updates

- 2025-11-19 Supports Gemini 3 Pro.
- 2025-11-12 Supports data synchronization from Confluence, S3, Notion, Discord, Google Drive.
- 2025-10-23 Supports MinerU & Docling as document parsing methods.
- 2025-10-15 Supports orchestrable ingestion pipeline.

- 2025-08-08 Supports OpenAI's latest GPT-5 series models.
- 2025-08-01 Supports agentic workflow and MCP.
- 2025-05-23 Adds a Python/JavaScript code executor component to Agent.
- 2025-05-05 Supports cross-language query.
- 2025-03-19 Supports using a multi-modal model to make sense of images within PDF or DOCX files.

## Stay Tuned

Star our repository to stay up-to-date with exciting new features and improvements! Get instant notifications for new releases!

## Key Features

### **\*\*"Quality in, quality out"\*\***

formats.

- [Deep document understanding](./deepdoc/README.md)-based knowledge extraction from unstructured data with complicated
- Finds "needle in a data haystack" of literally unlimited tokens.

### **\*\*Template-based chunking\*\***

- Intelligent and explainable.
- Plenty of template options to choose from.

### **\*\*Grounded citations with reduced hallucinations\*\***

- Visualization of text chunking to allow human intervention.
- Quick view of the key references and traceable citations to support grounded answers.

### **\*\*Compatibility with heterogeneous data sources\*\***

- Supports Word, slides, excel, txt, images, scanned copies, structured data, web pages, and more.

### **\*\*Automated and effortless RAG workflow\*\***

- Streamlined RAG orchestration catered to both personal and large businesses.
- Configurable LLMs as well as embedding models.
- Multiple recall paired with fused re-ranking.
- Intuitive APIs for seamless integration with business.

## System Architecture

## Get Started

### Prerequisites

- CPU  $\geq$  4 cores
- RAM  $\geq$  16 GB
- Disk  $\geq$  50 GB
- Docker  $\geq$  24.0.0 & Docker Compose  $\geq$  v2.26.1
- [gVisor](https://gvisor.dev/docs/user\_guide/install/): Required only if you intend to use the code executor (sandbox) feature of RAGFlow.

> [!TIP]

> If you have not installed Docker on your local machine (Windows, Mac, or Linux), see [Install Docker Engine](https://docs.docker.com/engine/install/).

### Start up the server

1. Ensure `vm.max\_map\_count`  $\geq$  262144:

> To check the value of `vm.max\_map\_count`:

>

> ```bash

> \$ sysctl vm.max\_map\_count

> ```

>

> Reset `vm.max\_map\_count` to a value at least 262144 if it is not.

>

> ```bash

> # In this case, we set it to 262144:

> \$ sudo sysctl -w vm.max\_map\_count=262144

> ```

>

> This change will be reset after a system reboot. To ensure your change remains permanent, add or update the

> `vm.max\_map\_count` value in `*/etc/sysctl.conf` accordingly:

>

> ```bash

> vm.max\_map\_count=262144

> ```

## 2. Clone the repo:

```
$ git clone https://github.com/infiniflow/ragflow.git
```

### 3. Start up the server using the pre-built Docker images:

> [!CAUTION]

> All Docker images are built for x86 platforms. We don't currently offer Docker images for ARM64.

> If you are on an ARM64 platform, follow [this guide]([https://ragflow.io/docs/dev/build\\_docker\\_image](https://ragflow.io/docs/dev/build_docker_image)) to build a Docker image compatible with your system.

> The command below downloads the `v0.22.1` edition of the RAGFlow Docker image. See the following table for descriptions of different RAGFlow editions. To download a RAGFlow edition different from `v0.22.1`, update the `RAGFLOW\_IMAGE` variable accordingly in **\*\*docker/.env\*\*** before using `docker compose` to start the server.

```
$ cd ragflow/docker
```

```
# git checkout v0.22.1
```

```
# Optional: use a stable tag (see releases: https://github.com/infiniflow/ragflow/releases)
```

```
# This step ensures the entrypoint.sh file in the code matches the Docker image ve
```

```
# Use CPU for DeepDoc tasks:
```

```
$ docker compose -f docker-compose.yml up -d
```

```
# To use GPU to accelerate DeepDoc tasks:
```

```
# sed -i '1i DEVICE=gpu' .env
```

```
# docker compose -f docker-compose.yml up -d
```

> Note: Prior to `v0.22.0`, we provided both images with embedding models and slim images without embedding models. Details as follows:

| RAGFlow image tag | Image size (GB) | Has embedding models? | Stable?        |
|-------------------|-----------------|-----------------------|----------------|
| v0.21.1           | ≈9              | ✓                     | Stable release |
| v0.21.1-slim      | ≈2              |                       | Stable release |

> Starting with `v0.22.0`, we ship only the slim edition and no longer append the **\*\*slim\*\*** suffix to the image tag.

4. Check the server status after having the server up and running:

```
$ docker logs -f docker-ragflow-cpu-1
```

The following output confirms a successful launch of the system:

\* Running on all addresses (0.0.0.0)

> If you skip this confirmation step and directly log in to RAGFlow, your browser may prompt a `network anormal`  
 > error because, at that moment, your RAGFlow may not be fully initialized.  
 >  
 5. In your web browser, enter the IP address of your server and log in to RAGFlow.  
 >  
 > With the default settings, you only need to enter `http://IP\_OF\_YOUR\_MACHINE` (\*\*sans\*\* port number) as the default  
 > HTTP serving port `80` can be omitted when using the default configurations.  
 >  
 6. In [service\_conf.yaml.template](./docker/service\_conf.yaml.template), select the desired LLM factory in `user\_default\_llm` and update the `API\_KEY` field with the corresponding API key.  
 > See [llm\_api\_key\_setup](https://ragflow.io/docs/dev/llm\_api\_key\_setup) for more information.  
 >  
 \_The show is on!\_

## Configurations

When it comes to system configurations, you will need to manage the following files:

- `MINIO_PASSWORD``.
- [env](./docker/.env): Keeps the fundamental setups for the system, such as `SVR\_HTTP\_PORT`, `MYSQL\_PASSWORD`, and
- [service\_conf.yaml.template](./docker/service\_conf.yaml.template): Configures the back-end services. The environment variables in this file will be automatically populated when the Docker container starts. Any environment variables set within the Docker container will be available for use, allowing you to customize service behavior based on the deployment environment.
- [docker-compose.yml](./docker/docker-compose.yml): The system relies on [docker-compose.yml](./docker/docker-compose.yml) to start up.

> The [./docker/README](./docker/README.md) file provides a detailed description of the environment settings and service  
 > configurations which can be used as `\${ENV\_VARS}` in the  
 [service\_conf.yaml.template](./docker/service\_conf.yaml.template) file.

To update the default HTTP serving port (80), go to  
 [docker-compose.yml](./docker/docker-compose.yml) and change `80:80`  
 to `:80`.

Updates to the above configurations require a reboot of all containers to take effect:

```
> `` `bash
> $ docker compose -f docker-compose.yml up -d
> `` `
```

Switch doc engine from Elasticsearch to Infinity

RAGFlow uses Elasticsearch by default for storing full text and vectors. To switch to [Infinity](https://github.com/infiniflow/infinity/), follow these steps:

1. Stop all running containers:

```
$ docker compose -f docker/docker-compose.yml down -v
```

> [!WARNING]

> `-v` will delete the docker container volumes, and the existing data will be cleared.

2. Set `DOC\_ENGINE` in **\*\*docker/.env\*\*** to `infinity`.

3. Start the containers:

```
$ docker compose -f docker/docker-compose.yml up -d
```

> [!WARNING]

> Switching to Infinity on a Linux/arm64 machine is not yet officially supported.

## Build a Docker image

This image is approximately 2 GB in size and relies on external LLM and embedding services.

```
git clone https://github.com/infiniflow/ragflow.git
cd ragflow/
docker build --platform linux/amd64 -f Dockerfile -t infiniflow/ragflow:nightly .
```

## Launch service from source for development

1. Install `uv` and `pre-commit`, or skip this step if they are already installed:

```
pipx install uv pre-commit
```

2. Clone the source code and install Python dependencies:

```
git clone https://github.com/infiniflow/ragflow.git
cd ragflow/
uv sync --python 3.12 # install RAGFlow dependent python modules
uv run download_deps.py
pre-commit install
```

3. Launch the dependent services (MinIO, Elasticsearch, Redis, and MySQL) using Docker Compose:

```
docker compose -f docker/docker-compose-base.yml up -d
```

Add the following line to `/etc/hosts` to resolve all hosts specified in **\*\*docker/.env\*\*** to `127.0.0.1`:

```
127.0.0.1          es01 infinity mysql minio redis sandbox-executor-manager
```

4. If you cannot access HuggingFace, set the `HF\_ENDPOINT` environment variable to use a mirror site:

```
export HF_ENDPOINT=https://hf-mirror.com
```

5. If your operating system does not have jemalloc, please install it as follows:

```
# Ubuntu
sudo apt-get install libjemalloc-dev
# CentOS
sudo yum install jemalloc
# OpenSUSE
sudo zypper install jemalloc
# macOS
sudo brew install jemalloc
```

6. Launch backend service:

```
source .venv/bin/activate
export PYTHONPATH=$(pwd)
bash docker/launch_backend_service.sh
```

7. Install frontend dependencies:

```
cd web
npm install
```

8. Launch frontend service:

```
npm run dev
```

\_The following output confirms a successful launch of the system:\_

9. Stop RAGFlow front-end and back-end service after development is complete:

```
kill -f "ragflow_server.py|task_executor.py"
```

## Documentation

- [Quickstart](https://ragflow.io/docs/dev/)
- [Configuration](https://ragflow.io/docs/dev/configurations)
- [Release notes](https://ragflow.io/docs/dev/release\_notes)
- [User guides](https://ragflow.io/docs/dev/category/guides)
- [Developer guides](https://ragflow.io/docs/dev/category/developers)
- [References](https://ragflow.io/docs/dev/category/references)
- [FAQs](https://ragflow.io/docs/dev/faq)

## Roadmap

See the [RAGFlow Roadmap 2025](https://github.com/infiniflow/ragflow/issues/4214)

## Community

- [Discord](https://discord.gg/NjYzJD3GM3)
- [Twitter](https://twitter.com/infiniflowai)
- [GitHub Discussions](https://github.com/orgs/infiniflow/discussions)

## Contributing

RAGFlow flourishes via open-source collaboration. In this spirit, we embrace diverse contributions from the community.

If you would like to be a part, review our [Contribution Guidelines](https://ragflow.io/docs/dev/contributing) first.